Welcome back to the learning series for the AWS Certified Machine Learning Engineer Associate exam. In the last three videos we finished Domain 2, where we chose models, trained them, and evaluated them. Now we move into Domain 3: Deployment and Orchestration of ML and AI Workflows.

Before we pick any AWS service, we need to understand how predictions will be used. I think about three questions. First, the request pattern. Does an application need an answer in milliseconds, like fraud checks at checkout? Or can it wait, like scoring a million customers overnight? Payload size matters too, because a large video file behaves very differently from a small row of features. Second, the model type. A small XGBoost model runs happily on CPU, while a large language model needs GPUs and lots of memory. RAG systems and agents add retrieval and tool calls on top. Third, the traffic shape. Steady traffic, sudden spikes, and long idle periods each point to different hosting options. On the exam, the scenario usually hides these clues in the question, so read for latency, payload size, and traffic pattern first.

Amazon SageMaker AI gives us four main inference options, and this is one of the most tested topics in this domain. Real-time endpoints run on instances that are always on. They give the lowest, most consistent latency, so they fit interactive applications with steady traffic. Serverless inference removes instance management. It scales with requests and you pay per use, which is great for traffic that comes and goes. The trade-off is cold starts after idle periods. Asynchronous inference puts requests in a queue and writes results to Amazon S3. It handles large payloads, up to one gigabyte, and long processing times, up to one hour, and it can scale down to zero when the queue is empty. Batch transform has no endpoint at all.

Running one endpoint per model gets expensive quickly, so SageMaker AI offers ways to share infrastructure. Multi-model endpoints host many models behind one container. Models stay in Amazon S3 and are loaded into memory when they are first called. This is perfect when you have hundreds of similar models, for example one model per customer, and all of them use the same framework.

For foundation models, the big decision is managed or self-hosted. Amazon Bedrock is the managed path. You call foundation models through an API with no servers to manage. On-demand mode charges per token, which fits variable usage. Provisioned Throughput reserves model capacity for steady, high-volume workloads, and it is often required to serve custom fine-tuned models. SageMaker AI is the self-hosted path. You can deploy models from SageMaker JumpStart or any open-weight model, choose the exact GPU instance type and count, and control the serving container and settings

Many teams train models somewhere else, on-premises or in another cloud, and want to serve them on AWS. The process follows five steps. First, package the model artifacts, usually as a model dot tar dot gz file stored in Amazon S3. Second, pick a container. If your framework is supported, use an AWS prebuilt framework container. If not, build your own image and push it to Amazon Elastic Container Registry. Third, import. For SageMaker AI, you create a model object that links the artifact and the container. For supported foundation model architectures, you can

use Amazon Bedrock Custom Model Import instead. Fourth, deploy by creating an endpoint configuration and an endpoint. Fifth, validate. Make sure the outputs match what you saw before migration, and check latency and cost. The key idea is that SageMaker AI does not care where the model was trained. It only needs the artifacts and a compatible container.

Generative AI deployments are more than a model behind an endpoint. Agents plan steps and take actions. With Amazon Bedrock Agents, you write instructions that define the agent's job and add action groups, usually backed by AWS Lambda or an API, so the agent can do real work like checking an order. Agents also need to talk to tools and to other agents. The Model Context Protocol, or MCP, is a standard way to expose tools and data to a model. Agent-to-agent protocols, like A2A, let separate agents collaborate. For RAG systems, deployment settings strongly affect quality.

Let's wrap up Part 1 with a checklist you can use on exam questions and in real projects. One: define the latency target, payload size, and traffic pattern. Two: use those to pick real-time, serverless, asynchronous, or batch inference. Three: decide whether models should share an endpoint, using multi-model endpoints, multi-container endpoints, or inference components. Four: for foundation models, choose Amazon Bedrock when you want managed simplicity, or SageMaker AI when you need control. Five: for agents and RAG, plan the tools, protocols, and retrieval settings. And six: validate quality, latency, and cost before going live. The overall rule is the same one we used for model selection: choose the lightest infrastructure that meets the requirement. In Part 2, we will look at how to provision and configure these resources. Thanks for watching, and see you in the next video.